ARCHITECTURE.md — Squad, explained simply

> A reference doc for the Squad app. Two sections:
>
> 1. **The full architecture** — what's in the system, who does what, how a request flows through it.
> 2. **The technologies, explained like you're 10** — what each piece (Next.js, Supabase, Clerk, etc.) actually is and why we picked it.

Part 1 — The full architecture

The problem we're solving

Every weekend, your friend group has the same conversation on WhatsApp: "anyone free? where do we go? what time?" Three hours of messages, decisions get lost, someone always asks "wait are we still on?". The conversation is messy because it's a chat — words flowing past each other — when what you actually need is a **structured answer**: a plan, a vote count, a yes/no.

Squad replaces the messy conversation with a structured object. Anyone proposes a plan. Friends vote In/Out/Maybe. The answer is always visible at the top of the screen. WhatsApp keeps being where you talk; Squad becomes where you decide.

The system, drawn out

```
┌──────────────────────────────────────────────────────────────┐
│                        USER'S PHONE                          │
│   Browser (Next.js app at squad.vercel.app)                  │
│   - React UI                                                 │
│   - Optimistic state for instant interactions                │
│   - WebSocket subscription for live vote updates             │
└──────────────────────────────────────────────────────────────┘
              |                  |                  |
              | HTTPS            | HTTPS            | WebSocket
              | (page loads,     | (sign-in,        | (live
              |  server actions) |  sessions)       |  updates)
              ▼                  ▼                  ▼
   ┌──────────────────┐  ┌──────────────┐  ┌──────────────────┐
   |  Vercel          |  |   Clerk      |  |  Supabase        |
   |  (Next.js        |  |  (auth)      |  |  Realtime        |
   |   serverless)    |  |              |  |                  |
   |                  |  |  - Google    |  |  - watches       |
   |  - Pages         |  |    OAuth     |  |    Postgres      |
   |  - Server        |  |  - Session   |  |  - broadcasts    |
   |    actions       |  |    cookies   |  |    changes       |
   |  - Webhook       |  |  - Webhook   |  |                  |
```

```
    |    receiver      |  |   →Squad   |  |                  |
    └──────────────────┘  └────────────┘  └──────────────────┘

              |                  |                  |
              | pooled           | webhooks         | subscribes
              | Postgres         | (user events)    | to table
              | via Drizzle      |                  | changes
              ▼                  ▼                  |
    ┌──────────────────────────────────────────┐   |
    |  Supabase Postgres (the database)        |◄──┘
    |                                          |
    |  7 tables:                               |
    |  users, circles, memberships, invites,   |
    |  plans, votes, comments                  |
    └──────────────────────────────────────────┘

              |
              | on plan create / comment add
              | → triggers email send (M7)
              ▼
    ┌──────────────────────────┐
    |  Resend                  |
    |  (transactional email)   |
    |  → sends to friends      |
    └──────────────────────────┘
```

Who does what

Component	Job	When it runs
Browser (Next.js client)	Renders UI, handles taps and types, makes optimistic updates when you vote	Whenever a user has the page open
Vercel (Next.js serverless)	Serves pages, handles server actions (mutations), receives Clerk webhooks	On every page load and every action (vote, create plan, etc.)
Clerk	Identity. Owns "who is this person?" Handles Google sign-in, session cookies, sends Squad a webhook when users sign up/update/delete	On signin, on every authenticated request, when a user is created
Supabase Postgres	The database. Every plan, vote, comment, invite, circle, user lives here	On every read or write the app does
Supabase Realtime	Watches Postgres for changes and broadcasts them via WebSocket. Makes the app feel "live"	Continuously, while users have the app open
Resend	Sends transactional emails (new plan notification, comment notification)	Triggered by server actions in M7+

How a single request flows through the system

Walking through one real interaction tells you everything about how the system works.

Scenario: Saturday 5pm. Karan creates a plan. Ravi votes on it from his phone.

Step 1. Karan opens Squad at `squad.vercel.app` on his phone. His browser asks Vercel for the page. Vercel runs a serverless function that:

- Reads Karan's session cookie
- Asks Clerk: "is this cookie valid?"
- Clerk says yes, returns Karan's userId (`user_2nQX...`)
- Vercel queries Postgres via Drizzle: "what circles is this user in? what plans exist in those circles?"
- Vercel renders the HTML with Karan's circle home and streams it to his browser

Step 2. Karan taps "+ New Plan", fills out the form (Dinner at Bawarchi, 8:30pm), hits Submit.

Step 3. The form has a **server action** attached — a special Next.js pattern that lets a button on the page directly call a function on the server. Karan's browser POSTs to Vercel.

Step 4. The server action:

- Re-validates Karan's session via Clerk
- Calls `requireMembership(circleId, karanId)` to confirm he's in this circle
- Inserts a row into the `plans` table via Drizzle
- Inserts Karan's vote as 'in' (creator auto-votes)
- Returns the new plan ID

Step 5. Karan's browser navigates to the plan detail page. He sees his plan with 1 vote (himself).

Step 6. Meanwhile, the moment Postgres committed the new plan row, **Supabase Realtime** noticed. Every other friend in the circle who has the app open has an active WebSocket connection. Their browsers receive the change event. React re-renders. "Dinner at Bawarchi" appears at the top of their screen. They didn't refresh. It just appeared.

Step 7. Ravi sees the email (in M7+) on his phone, taps "Vote now."

Step 8. Ravi isn't signed in (it's been a week). Squad redirects him through Clerk → he signs in with Google → lands back on the plan page.

Step 9. Ravi taps the green "🟢 In" button. The UI updates instantly — green button highlights, vote count goes from 1 to 2. **No network roundtrip happened yet.** This is **optimistic UI** — we're betting the server will accept the vote.

Step 10. In parallel, a server action POSTs to Vercel: vote(planId, status='in'). Drizzle UPSERTs the votes table.

Step 11. Postgres emits a change event. Supabase Realtime broadcasts. Karan's browser, still open on the plan page, receives the event. The vote count updates from 1 to 2 with Ravi's avatar appearing. Karan didn't refresh either.

This is what makes the app feel collaborative. Eight friends can be on the same plan at the same time, voting and commenting, and everyone sees changes within ~500 milliseconds.

The 7 database tables, in plain English

Table	What it stores	Example row
users	Everyone who's ever signed up	`id: user_2nQX...` (from Clerk), `email: shivam@...`, `display_name: Shivam`, `avatar_url: https://...`
circles	Friend groups	`id: <uuid>`, `slug: hyderabad-crew`, `name: Hyderabad Crew`, `created_by: user_2nQX...`
memberships	Which user is in which circle, with what role	`user_id: user_abc, circle_id: <uuid>, role: admin`
invites	Active invite codes for joining circles	`code: x9k2mq8z, circle_id: <uuid>, max_uses: null, uses: 3`
plans	Proposed events (dinner, chai, movie)	`title: Dinner at Bawarchi, type: eat, starts_at: 2026-04-26 20:30, location: Bawarchi`
votes	Each user's vote on each plan	`user_id: user_abc, plan_id: <uuid>, status: in`
comments	Discussion thread under each plan	`plan_id: <uuid>, user_id: user_abc, body: cool, count me in`

Why these architectural choices

Why Postgres + Drizzle instead of Supabase's REST API? Supabase offers a JS client that lets you do `supabase.from('plans').select(...)` directly from the browser. Convenient, but security relies on Postgres Row-Level Security policies — complex SQL rules where one mistake leaks data. Instead we use Drizzle ORM with a privileged connection: auth and authorization happen in TypeScript inside Next.js server actions. More code, but easier to audit. For a 12-person friend app, simpler beats fancier.

Why Clerk instead of Supabase Auth? Supabase has built-in auth which would mean one fewer service. But Clerk's Google sign-in works in 5 minutes, the React components are polished, the dashboard is great. We pay the cost of mirroring users from Clerk → our `users` table via webhook, but it's a well-trodden pattern.

Why server actions instead of API routes? Server actions are faster to write, type-safe end-to-end, and more secure (no exposed endpoint). The only API route we have is the Clerk webhook, because Clerk has to POST to a real URL externally.

Why optimistic UI for voting? On 4G, a network roundtrip takes 200-500ms. Without optimistic updates, voting feels sluggish. Voting is the most-frequent interaction, so it has to feel instant.

Why limit to 8 routes? Every route is something to build, test, and teach friends. Most apps don't fail because they had too few features — they fail because they had too many half-built ones.

Part 2 — The technologies, explained like you're 10

Next.js

Imagine you have a paper book. Every page is already printed. When you flip to page 50, the page is already there.

Now imagine a magic book where pages appear as you flip to them. The book asks "what page does this person want?" Then a tiny invisible printer prints that page on demand, just for them, right when they need it.

That's Next.js. It's a website-making framework where pages can be either pre-printed (fast, like a normal book) or printed-on-demand (custom for each person, like a personalized newspaper). It's built on top of **React**, which is basically a way to describe what UI should look like using JavaScript.

In Squad, every page that shows your circle's plans is "printed on demand" because it shows YOUR plans, not a generic list. When Karan opens the app, Next.js asks the database "what plans are in Karan's circle?" then prints the page just for him.

Why we picked it: Best fit for full-stack web apps in 2026. Has a feature called **server actions** (more on this below) that makes building data-mutation forms much faster.

Vercel

If Next.js is the magic book, Vercel is the bookstore that hosts the magic book.

Vercel is a hosting service. You give them your code, they put it on the internet, they run the magic-printer-on-demand whenever someone visits your site. They also do a clever trick: when you push new code to GitHub, they automatically build a new copy of your site within 90 seconds. Every change you make goes live almost immediately.

Vercel uses something called **serverless functions**. Instead of running a computer 24/7 waiting for visitors (expensive), they spin up a tiny computer for each request, run the request, and then turn the computer off. Like having an Uber driver instead of owning a car: only pay when you're using it.

Why we picked it: Free tier handles a 12-person friend app for years. Made by the same company as Next.js so the integration is seamless.

Supabase

Supabase is two things smushed into one product:

1. A database (Postgres). Think of a database as a giant, very organized spreadsheet. It has tables (like sheets), rows (like records), and columns (like fields). When the app says "save this plan," the data goes into a row in the `plans` table. When the app says "show me all plans for this circle," Postgres finds the matching rows and returns them.

Postgres specifically is a 30-year-old database technology that's still considered one of the best in the world. It's reliable, fast, and supports basically any feature you'd ever need. We're using it through Supabase, which hosts and manages it for us so we don't have to.

2. Realtime. This is the cool part. Imagine the database has a little assistant standing next to it, watching everything that happens. When someone adds a new plan, the assistant whispers to every browser that's currently looking at the app: "psst, new plan, here are the details." The browsers update without anyone hitting refresh.

Technically this is done via **WebSockets** — a kind of phone line that stays open between the browser and the server. Different from regular page loads (which are like sending a postcard, getting one back, and being done). A WebSocket is a continuous open call where both sides can shout new info to each other any time.

Why we picked it: Free tier is generous. Database + realtime + storage + auth all in one product. We're using database and realtime; skipping their auth (using Clerk instead).

Clerk

Clerk is a "who are you?" service.

Imagine Squad is a clubhouse. Anyone can walk up to the door. But to come inside, you have to prove who you are. Clerk is the bouncer. When you walk up, Clerk says:

- "Click 'Sign in with Google.'"
- You go through Google's sign-in.
- Google tells Clerk "yes, this is Shivam, here's his email and photo."
- Clerk gives you a wristband (a session cookie) that proves "I checked, this is Shivam."

Now whenever you do anything in the app, the bouncer just glances at your wristband and says "okay come in." Until the wristband expires (usually weeks), you don't have to sign in again.

Clerk also keeps a record of every user it's ever bouncered for. When a new user signs up, Clerk fires off a notification (a **webhook**) to Squad: "hey, new person just got a wristband, you should know about them." Our webhook handler in Vercel receives that notification and creates a row in our `users` table so we can track plans/votes/comments by them.

Why we picked it: Setting up Google sign-in by yourself is annoying — you'd have to handle OAuth flows, session cookies, password resets if you added passwords, "remember me," etc. Clerk does all of it in 5 minutes. Free tier covers a friend app forever.

Drizzle (ORM)

Imagine you speak English but the database speaks SQL (Structured Query Language). To ask the database "give me all plans in this circle," you'd have to say:

```
SELECT * FROM plans WHERE circle_id = '...' ORDER BY starts_at DESC;
```

That's SQL. It's a real language. Writing it by hand for every query is tedious and error-prone — typos cause bugs that only show up at runtime.

Drizzle is a **translator**. You write code in TypeScript like:

```typescript
db.select().from(plans).where(eq(plans.circleId, circleId)).orderBy(desc(plans.start
```

Drizzle translates that into the SQL above and sends it to Postgres. The benefits:

- Your code editor knows what tables and columns exist (autocomplete!)
- If you typo a column name, your code editor tells you immediately, not at runtime
- Refactoring is easier — rename a column, the editor finds every place that needs updating

This kind of tool is called an **ORM** — Object-Relational Mapper. Drizzle is a specific ORM that's lighter and more transparent than older ones like Prisma or Sequelize.

Why we picked it: Type-safe, less magic than alternatives, and works really well with serverless connections (which our Vercel functions use).

TypeScript

JavaScript is the language browsers speak. It's flexible, but it's also infamous for letting you make stupid mistakes. Like, you can write `user.emial` (typo'd "email") and JavaScript happily runs the code, returns `undefined`, and you find out hours later when your app silently breaks.

TypeScript is JavaScript with a safety helmet. You declare the shape of things:

```typescript
type User = { id: string; email: string; displayName: string; }
```

Now if you write `user.emial`, your editor underlines it in red before you even save. Bugs caught at write-time instead of runtime.

The cost: a little more typing upfront. The benefit: catastrophically fewer bugs in production. For an app where bugs annoy your friends, this is worth it.

We use it in strict mode, which means every variable must have a known type. No "any" allowed. Drizzle generates TypeScript types from your database schema, so the database tables flow into

your code automatically.

Tailwind CSS

CSS is how you style web pages. Traditionally you'd write:

```
.my-button {
  background: blue;
  color: white;
  padding: 8px 16px;
  border-radius: 4px;
}
```

Tailwind says: instead of inventing class names like `.my-button` and writing CSS rules, just put the styles directly on the element using pre-named utility classes:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">Click me</button>
```

Each class does one specific thing: `bg-blue-500` = blue background, `text-white` = white text, `px-4` = horizontal padding, `py-2` = vertical padding, `rounded` = rounded corners.

Sounds verbose, but in practice it's way faster because you never leave your component file to go edit a separate stylesheet, and you never have to invent class names. Most teams using Tailwind ship UI 2-3x faster than teams writing custom CSS.

Why we picked it: Standard for shipping fast in 2026. We used it on the blog already.

shadcn/ui

If you've ever used a website, you've seen buttons, dropdowns, dialogs, dropdown menus. You probably don't think about them — they just work. But building a polished button from scratch (handling hover, focus, disabled, loading, dark mode, accessibility) takes a surprising amount of work.

shadcn/ui is a collection of pre-built React components — buttons, dialogs, forms, dropdowns — that look good out of the box. Instead of installing them as a library, you copy their code into your project, so you can customize them later if needed.

It's the "I want polished UI now, I'll worry about uniqueness later" choice. For Squad, where speed > beauty, this is exactly right.

Why we picked it: Different priority from the blog. The blog needed distinctive design because it's a presentation surface. Squad needs functional, fast UI — friends will tap buttons, vote, leave comments. Generic-but-good is better than custom-but-slow-to-build.

Resend

Imagine you want to send 12 friends a custom email when a new plan is created. You can't just open Gmail and BCC them — you'd be doing it manually every time. You need to send emails programmatically.

Most email servers (the kind your inbox uses) treat unknown senders as spam. If your app tries to send email directly from `squad@yourcomputer.com`, it'll go straight to spam folders.

Resend is a service that sends email on your behalf, with all the deliverability infrastructure that makes the email actually arrive in inboxes (DKIM signatures, SPF records, dedicated IP reputation, bounce handling, you don't need to know what these are — Resend handles them).

You give Resend your API key, a list of recipients, and an HTML email template. Resend sends them. Done.

Why we picked it: Cheapest dev-friendly email service. Free tier of 3000 emails/month is far more than a 12-person friend app needs (probably 50/month). Modern API, good docs.

We're not using it until M7. For now we just have the env var ready.

react-hook-form + zod

Forms are deceptively annoying. You have to:

- Track what the user typed in each field
- Validate it (is the email valid? is the slug 3-40 chars?)
- Show error messages without scaring the user
- Submit to the server
- Show loading state while waiting
- Handle server errors and show them nicely
- Reset the form on success

Doing all of this by hand for every form is tedious. **react-hook-form** is a library that handles the state-tracking and submission for you. **zod** is a library that handles validation — you describe what valid input looks like:

```typescript
const createCircleSchema = z.object({
  name: z.string().min(3).max(40),
  slug: z.string().regex(/^[a-z0-9]+(-[a-z0-9]+)*$/),
});
```

Zod automatically validates user input against this schema and produces friendly error messages. The validation works on both the client (instant feedback as the user types) and the server (security — never trust the client). Same schema, two places.

Why we picked them: Standard combo for forms in modern React apps. Saves a lot of repetitive code.

sonner

When something happens (you submitted a form, you voted, an error occurred), you want to show a small notification at the bottom or top of the screen — "Vote saved" or "Couldn't save vote, try again." These are called **toasts**.

Sonner is a tiny library that shows toasts beautifully without you thinking about it. Call `toast.success("Saved")` and a green toast slides in, sits for 3 seconds, slides out.

Why we picked it: Recommended by shadcn/ui. Looks polished, no configuration needed.

Server actions (Next.js 15 feature)

Traditionally, when you submit a form on a website, the browser sends an HTTP POST to a server endpoint like `/api/create-plan`, the server runs code, sends a response, the browser reads it.

You have to write the endpoint AND the form. Two pieces, both have to know about the same data shape.

Server actions are a new pattern where you write a single function:

```typescript
'use server';
async function createPlan(input: { title: string; type: string }) {
  // ... write to database
}
```

And use it directly from a form:

```tsx
<form action={createPlan}>
  <input name="title" />
  <button type="submit">Create</button>
</form>
```

Behind the scenes, Next.js wires up the HTTP POST and the parsing. You don't see the plumbing. The function "looks" local but actually runs on the server with database access.

This is faster to write, type-safe end-to-end (no JSON parsing), and more secure (no exposed endpoint to attack from outside).

Why we use them: Less code than API routes, fewer bugs, faster to ship.

Optimistic UI

When you tap "Vote In," you have two choices:

Pessimistic: UI shows a loading spinner. Send vote to server. Wait for response (200ms-500ms). Update UI when response arrives.

Optimistic: Update UI immediately as if the vote succeeded. Send vote to server in parallel. If server says "actually no, that failed," undo the UI change and show an error.

Optimistic feels much faster — instant. The risk is rare error cases where you have to "undo" the visual change. For voting, where 99.9% of votes succeed, optimistic is the obvious choice.

This is a UI pattern, not a library. We implement it ourselves in M5 using React's state and Next.js's server actions.

WebSockets and Realtime

A normal browser-to-server connection is like sending a postcard. You write a request, mail it, get a postcard back, the conversation ends. To get more info, you have to send another postcard.

WebSockets are like a phone call that stays open. Once connected, both sides can shout new info to each other any time, instantly, until someone hangs up.

In Squad, every browser opens a WebSocket to Supabase Realtime when the user lands on a circle page. Realtime is "subscribed" to changes on the votes and comments tables. When Postgres commits a new row, Realtime broadcasts the change over every open WebSocket. Every browser receives it within ~100ms. React updates the UI.

This is what makes the app feel "alive" — votes appearing, comments appearing, all without anyone hitting refresh.

pnpm

When you write code, you use **packages** other people wrote — Next.js, React, Tailwind, etc. To install them, you use a **package manager**. The default is `npm`, but `pnpm` is faster, uses less disk space, and is stricter about dependencies (which catches bugs).

It's a command-line tool. You type `pnpm install some-package` and it gets added to your project.

Why we picked it: Faster + more disk-efficient. Same as the blog.

git and GitHub

git is a tool that tracks changes to your code. Every "commit" is a snapshot of what your project looked like at that moment. You can go back to any snapshot, see what changed between any two snapshots, and most importantly — share snapshots with other people and computers.

GitHub is a website that hosts your git snapshots online. When you push your commits to GitHub, they become accessible from anywhere — your laptop, your phone, your Vercel deployment.

Vercel watches your GitHub repo. Every push to `main` triggers a fresh deployment.

Why we use it: It's the universal default for code. Even solo, the snapshot history is invaluable when you break something and need to get back to a working state.

Quick reference: which service does what

When something breaks, knowing which service to look at first saves debugging time.

Symptom	Service to check first
User can't sign in	Clerk dashboard → check user records, webhook delivery
User signed in but no row in our `users` table	Clerk webhook delivery history
Database query failing	Supabase dashboard → check Tables, RLS policies, connection limits
Vote happened but other users don't see it update	Supabase Realtime → check publication includes the table
Page is slow or returning 500	Vercel function logs
Email not arriving	Resend dashboard → check delivery logs, bounce reports
Code change not deployed	GitHub → verify push went through; Vercel → check Deployments tab

---

Things this architecture deliberately doesn't have (and why)

These are common features in bigger apps that we skipped intentionally.

Feature	Why we skipped
Custom auth (passwords, magic links)	Google sign-in is enough for 12 friends. Adding more is more code, more support burden, more security risk.
Real observability (Sentry, PostHog)	At 12 users, friends will tell you when things break. Logs in Vercel are enough.
Background jobs / queues	We don't have any work that takes long enough to need a queue.
Caching layer (Redis, Upstash)	Free tier Postgres handles 12 users without breaking a sweat.
Push notifications	Email handles "you forgot to check the app." Push requires service workers, browser permissions, mobile app considerations — a rabbit hole.
File uploads (avatars, photos)	Avatars come from Google via Clerk. Plan photos are a v2 wishlist item.
Multi-tenancy / orgs	Circles ARE the multi-tenancy. Each circle is invite-only.
Search	At 100 plans you can scroll. Search becomes useful at 1000+, which we won't hit.
Internationalization	App is for your friend group. Add it if Squad grows beyond.
Analytics dashboard	What you'd want to know (who voted what) is one Supabase query away.

When tempted to add any of the above before v1 ships, file in PLAN.md §13 (v2 wishlist) and move on.

The five things to keep in your head

1. **Three external services** — Clerk (auth), Supabase (database + realtime), Resend (email, M7+). Each has env vars, a dashboard, and webhooks.
2. **The users table is mirrored from Clerk via webhook.** If a user appears in Clerk but not in Postgres, the webhook is broken.
3. **Realtime requires the publication to include the table.** If votes update in DB but other users don't see them, the publication is missing the table.
4. **Server actions are the mutation pattern.** Every "do something" interaction (vote, create, comment) is a server action. They run on Vercel with DB and session access.
5. **Optimistic UI for interactions, server is source of truth.** UI updates first, DB updates second, Realtime fans out third. If any step fails, UI rolls back.

Print this page out and tape it to your monitor for the next 8 milestones.